

Automating Peak Readiness:

Building Workload Models from Observability Signals

How to derive RPS, concurrency, scenario weights, and test duration directly from production telemetry.



Vasuki Uday Kiran Vudathala

Staff Performance Engineer, AI/ML · ServiceNow

“Peak readiness should be computed, not negotiated.”

September 2026

The most expensive word in peak readiness is “estimate.”

A workload model that starts with assumptions can produce a perfectly executed test of the wrong workload.

MANUAL MODELING

13–22 hrs

2–4 people · multiple handoffs

AUTOMATED MODEL

< 5 min

1 engineer · reproducible queries

TRACEABILITY

Every number

maps back to telemetry

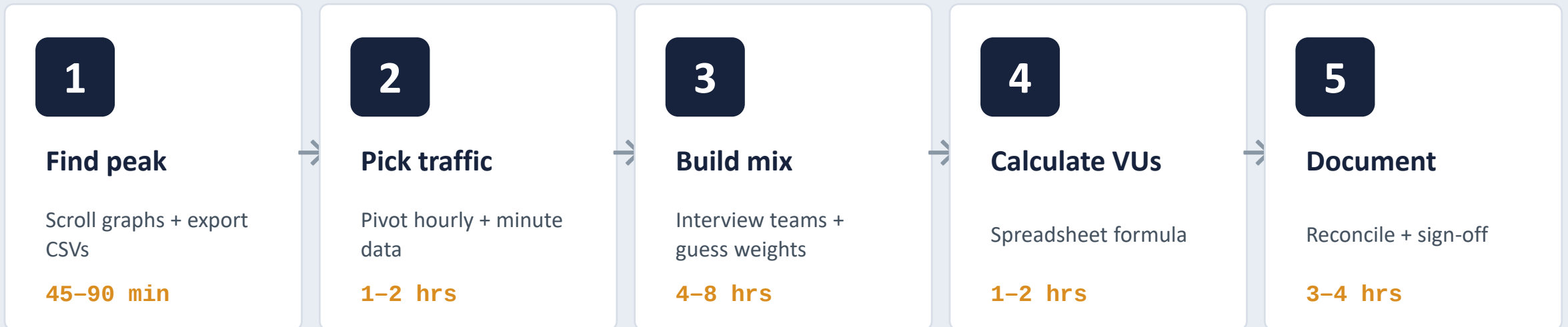
The goal is not simply speed.

The goal is to replace opinion with evidence — and keep the evidence attached to the model.

Published example: ~99% reduction in modeling effort per peak-readiness cycle.

Manual workload modeling is a chain of fragile decisions

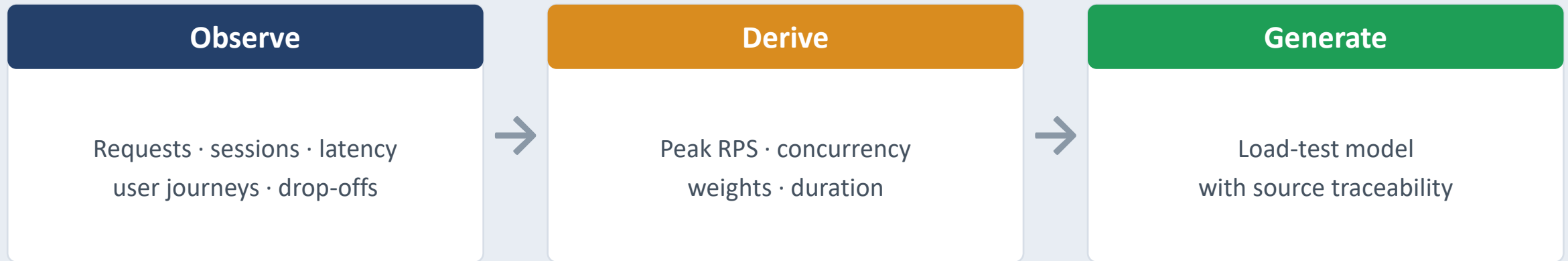
Each handoff adds delay. Each spreadsheet adds another place where the model can drift away from production.



The failure mode is subtle: **the test can be “green” while the workload model is wrong.**

Treat observability as the workload specification

The same telemetry used to diagnose production can also tell you what to reproduce before the next peak.



Not an APM feature. A methodology.

New Relic and Dynatrace are examples. Any platform exposing transaction + session telemetry can implement the same questions.

A six-step drill-down from history to a load-test model

Start broad, progressively narrow to the worst observed window, then reconstruct the workload around it.



Growth factor

Apply after extracting last year's peak. **1.2×** conservative **1.3×** moderate **1.5×** aggressive **2.0×** stress ceiling

Formula: $\text{Target RPS} = \text{Peak RPS} \times \text{Growth} \times \text{Safety Buffer (1.1)}$

Peak Day → Peak Hour → Peak Minute → Scenario Mix → Concurrent Users → Active Sessions

Don't size from an average. Find the worst minute.

The modeling error often begins when teams jump directly from a “busy day” to a single average RPS number.

1

PEAK DAY

Nov 28, 2025

152,840 requests

→ Anchor the analysis

2

PEAK HOUR

11:00–12:00

29,640 requests

→ Set the test window

3

PEAK MINUTE

11:23

4,920 requests

→ Derive 82 RPS

Peak minute = the load-test throughput baseline. Everything before it is just narrowing the search space.

Step 1: establish the correct reference day

The peak day comes from a direct query — reproducible and auditable, not eyeballed.

NRQL — NEW RELIC

Portable to any APM — same logic in Dynatrace DQL, etc.

```
SELECT count(*) AS apm_transactions, rate(count(*), 1 minute) AS avg_rpm, average(duration) AS avg_response_sec,
percentile(duration, 95) AS p95_response_sec, filter(count(*), WHERE error IS TRUE) AS error_count
FROM Transaction WHERE appName = 'your-app-name' AND transactionType = 'Web'
FACET dateOf(timestamp) SINCE '2025-11-15 00:00:00' UNTIL '2025-12-15 23:59:59' LIMIT 60
```

Peak day identified
Nov 28, 2025

Peak day requests
152,840

Unique sessions
28,410

Avg response (ms)
318 ms

Why this matters
All later queries inherit this exact date window.



QUERY RESULT TABLE — TOP 5 DAYS RANKED BY REQUEST VOLUME

Date	Total requests	Unique sessions	Avg resp (ms)	P95 (ms)	P99 (ms)
Nov 28, 2025	152,840	28,410	318	820	1,240
Nov 29, 2025 (Sat)	98,200	19,640	295	710	1,080
Dec 1, 2025	87,550	17,200	280	680	1,020
Nov 27, 2025	76,300	15,900	265	640	960
Dec 8, 2025	68,100	14,200	270	655	990

Step 2: the peak hour defines the sustained-load window

Bucket the peak day by hour — volume, sessions, and latency together mark the sustained window.

NRQL — NEW RELIC

```
SELECT count(*) AS requests_per_hour, rate(count(*), 1 minute) AS avg_rpm_within_hour,
average(duration) AS avg_response_sec, percentile(duration, 50, 95, 99) AS latency_pct
FROM Transaction WHERE appName = 'your-app-name' AND transactionType = 'Web'
TIMESERIES 1 hour SINCE '2025-11-28 00:00:00' UNTIL '2025-11-28 23:59:59'
```

Portable to any APM — same logic in Dynatrace DQL, etc.



Step 3: turn the peak minute into a forward-looking RPS target

Observe first. Then apply growth and safety explicitly — never bury them inside the baseline.

NRQL — NEW RELIC

Portable to any APM — same logic in Dynatrace DQL, etc.

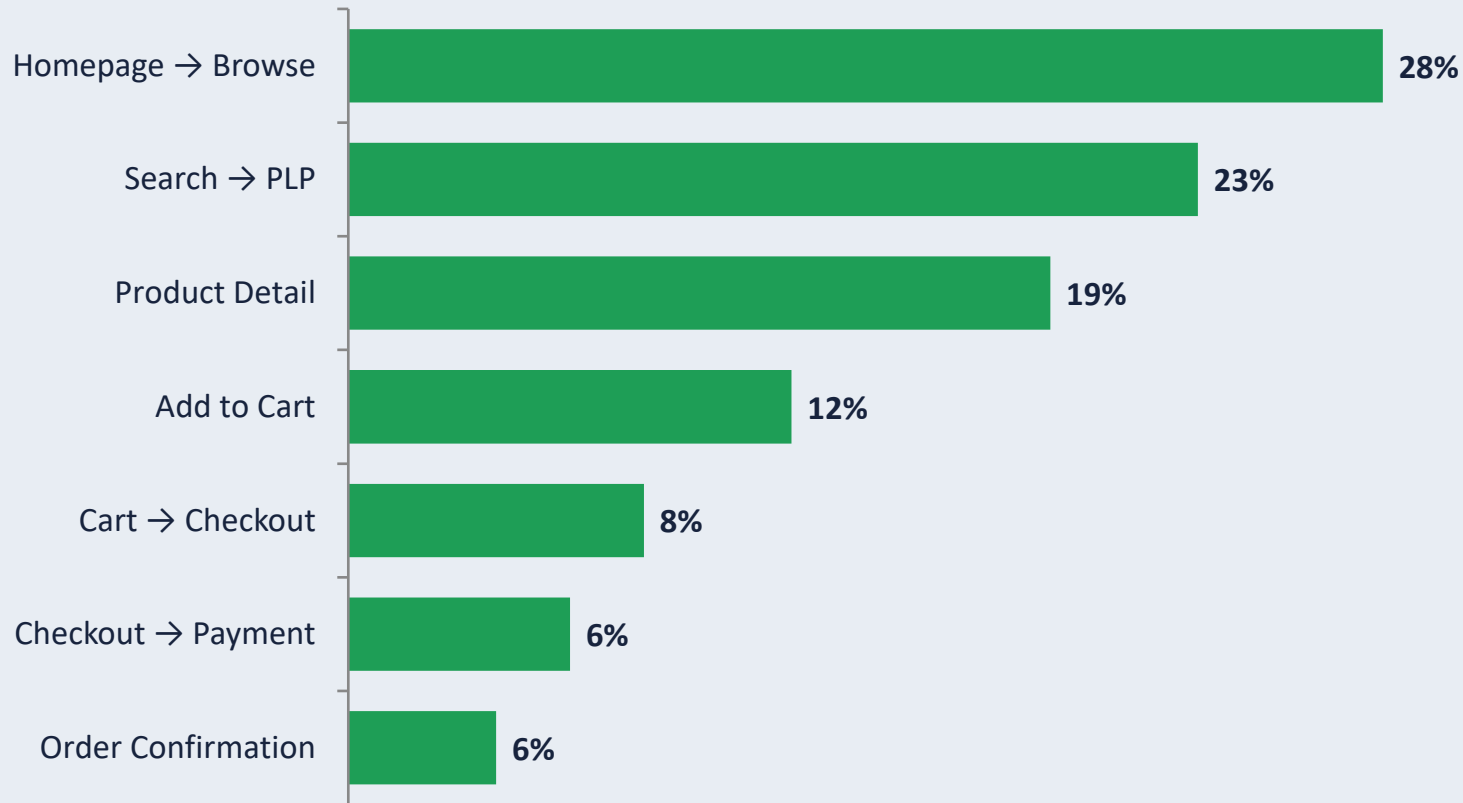
```
SELECT count(*) AS requests_per_minute, uniqueCount(session) AS active_sessions_per_min,  
rate(count(*), 1 second) AS rps, percentile(duration, 95) AS p95_latency  
FROM Transaction WHERE appName = 'your-app-name' AND transactionType = 'Web'  
TIMESERIES 1 minute SINCE '2025-11-28 11:00:00' UNTIL '2025-11-28 11:59:59'
```



$$82 \text{ observed RPS} \times 1.3 \text{ growth} \times 1.1 \text{ safety} = 114 \text{ RPS test ceiling}$$

Step 4: let telemetry set the scenario weights

Top transactions alone are not enough. Use browser and session telemetry to preserve the shape of real journeys.



Weight = observed share

not stakeholder preference.

Each percentage becomes a scenario's virtual-user weight in the load test — straight from production traffic.

THREE TELEMETRY ANGLES

- Transaction volume**
which server ops dominate
- Top routes / pages**
the real browser-side view
- Funnel drop-off**
where sessions actually leak

Step 5: size virtual users from session concurrency

A VU represents a user journey — different from the number of HTTP requests simultaneously in flight.

$$\text{Concurrent users} = (\text{hourly sessions} \times \text{avg session duration}) \div 3600$$

Worked example

$$4,280 \times 444 \text{ sec} \div 3600 = \mathbf{529 \text{ simultaneous users}}$$

APPLY 1.3× GROWTH

688 VUs simultaneously active

Transaction concurrency

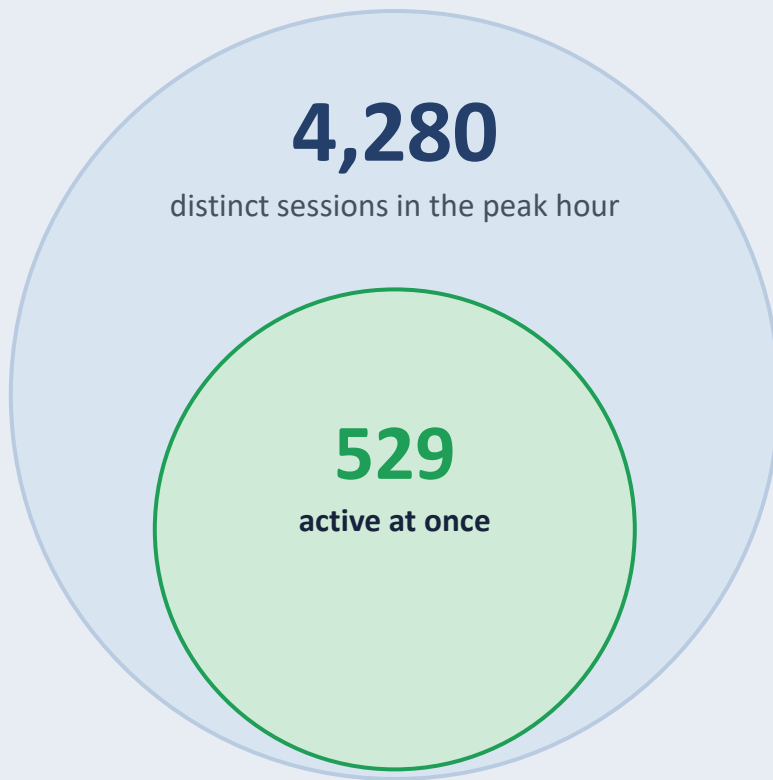
A separate metric — useful for infrastructure and thread-pool sizing, not for counting virtual users.

Little's Law answers “how many requests are in flight?”

Key distinction: session concurrency answers “how many users are navigating?” — that is what a VU represents.

Step 6: keep simultaneous concurrency and session population separate

They answer different questions. Conflating them is one of the easiest ways to oversize or undersize a test.



SIMULTANEOUS

529

Step 5 · concurrency

SESSION POPULATION

4,280

Step 6 · peak hour

GROWTH TARGET

688

concurrent VUs

GROWTH POOL

5,564

distinct sessions

Use concurrency to drive active VUs. Use the session population as a workload / allocation bound.

Reconstruct journeys, not just endpoints

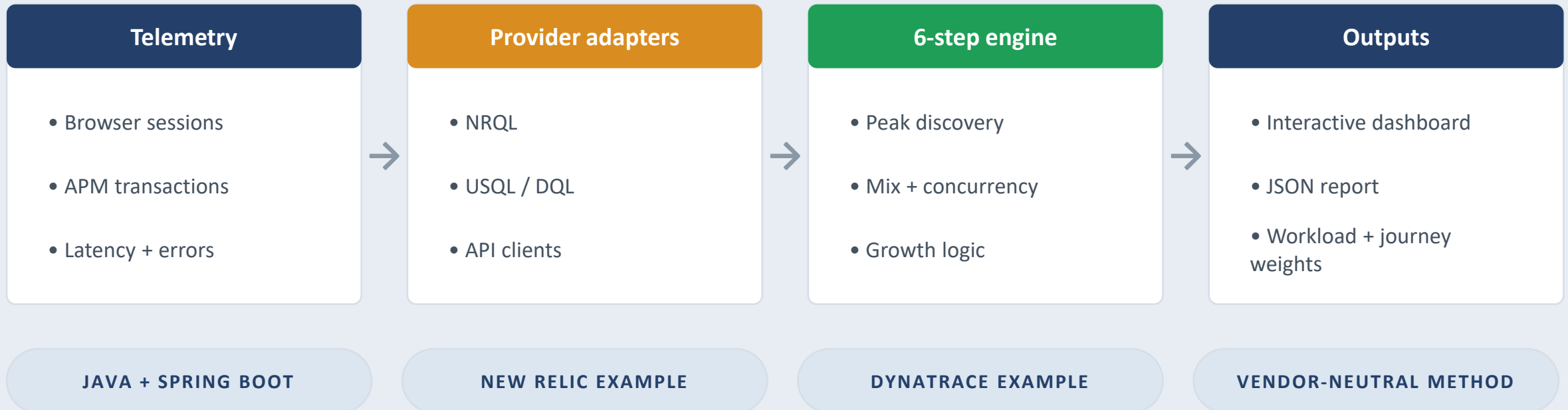
Session telemetry reveals where users enter, where they exit, and which drop-offs should become explicit test scenarios.



The “failed” journeys matter too: abandon, checkout drop, payment drop, browse exit, early exit — each becomes a weighted scenario.

Automate the method, not just the queries

A small analysis layer turns observability APIs into a repeatable readiness artifact.



The product is traceability: **query** → **derived number** → **test parameter**.

The model is small. The evidence behind it is not.

Every parameter below has a source query — which is what makes the model defensible.

Parameter	Observed	Model input
Peak day	Nov 28, 2025	reference window
Peak hour	11:00–12:00	60-minute test
Peak RPS	82 observed	107 growth · 114 ceiling
Concurrent users	529 observed	688 growth target
Session population	4,280 observed	5,564 growth pool
Top scenarios	7	telemetry-weighted

3 rules

- 1 Derive before you forecast.
- 2 Keep observed load separate from growth.
- 3 Make every test parameter traceable.



Full methodology + queries

How to Build an Automated Workload Model for Peak Readiness

Thank you

“Peak readiness should be computed, not negotiated.”

Questions?

Vasuki Uday Kiran Vudathala

Staff Performance Engineer, AI/ML · ServiceNow

LinkedIn linkedin.com/in/vasukiudaykiran

Method freeCodeCamp — Automated Workload Model for Peak Readiness



Scan to connect on LinkedIn